EIE330 FPGA-based System Design — Lab 2 Requirements

Lab 2 contains two tasks:

- **Lab 2-1 Counter**
- **Lab 2-2 Running Light**

Do **both** tasks. For **each** task, follow the development flow taught in the lectures from start to finish — preliminary research, FPGA solution, coding, functional simulation, time analysis, and testing on the development board — and write **one report per task**.

Both tasks are **sequential** designs: they use the **50 MHz system clock** and a **reset button**, and they contain **internal variables** (counters) whose values you must analyze, not only the inputs and outputs.

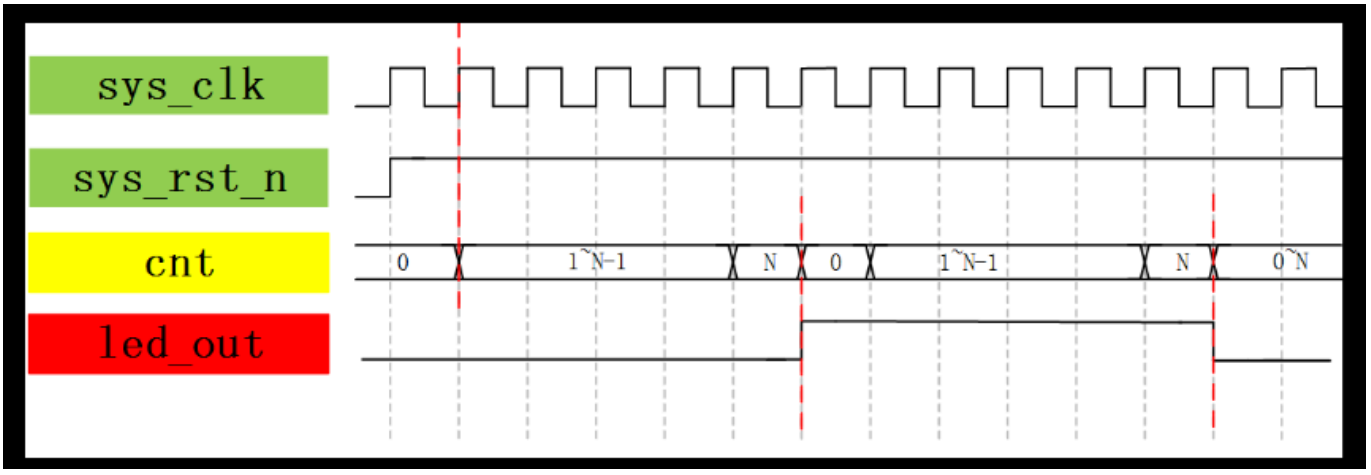
---

1. Task 2-1: Counter

Use the development board to implement a **counter that controls a flashing LED**. The LED flashes at **1 Hz**:

- at the beginning, the LED is **on**;
- after **0.5 seconds**, it turns **off**;
- after another **0.5 seconds**, it turns **on** again;
- this repeats forever.

The required signal timing is:



A **reset button** is required: whenever it is pressed, the system returns to the initial state **immediately** — the LED is on and the timing restarts from zero. This must work at **any** moment, including in the middle of a count, not only at convenient times.

Requirements:

- Use the **50 MHz** clock source of the board as the system clock, and count clock cycles to measure the time.

- Name your signals `sys_clk`, `sys_rst_n`, and `led_out`; the essential internal counter is `cnt` (you may add other internal variables if your design needs them).
- Use **LED0** (the `led1` signal in the simulator window) as the controlled LED.
- On the virtual board, map your reset signal to a **normal button** (e.g., `B3`). **Do not** map it to the `RESET` key: `RESET` restarts the whole simulation, so your reset logic could never be tested with it.

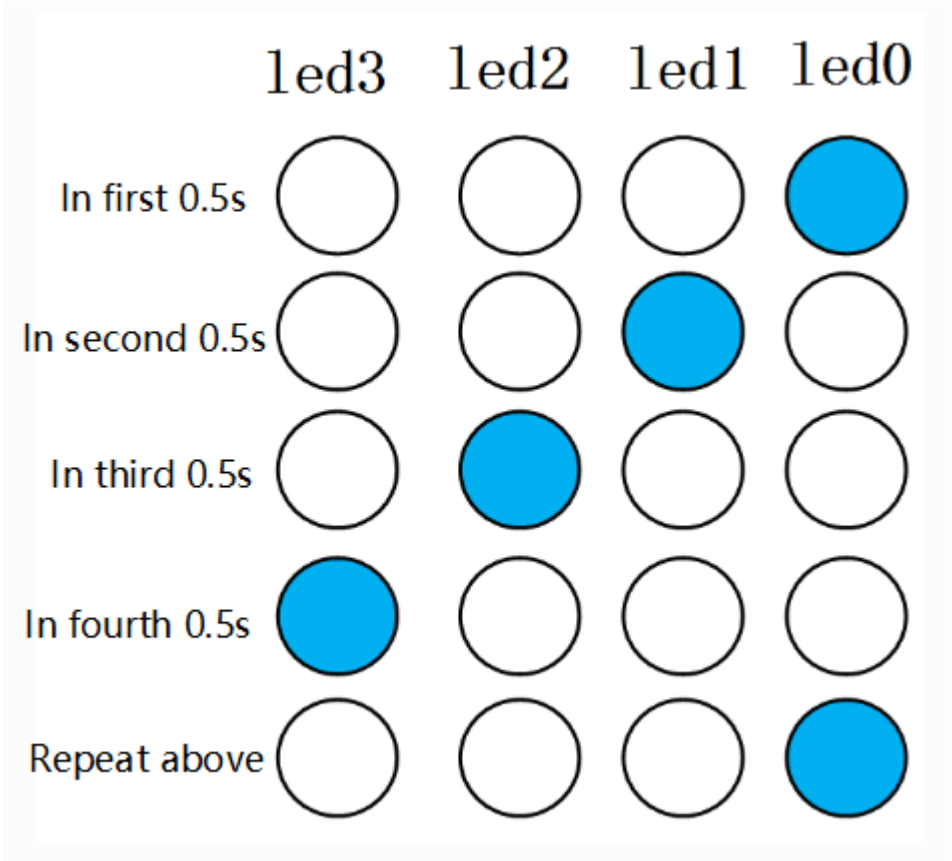
Verification: simulation **and** testing on the development board are both required (see Sections 3 and 4).

2. Task 2-2: Running Light

Use the development board to implement a **running light**: the LED lights arranged in a row flash **in turn**, like running water, and it keeps circulating.

- **4 LEDs**: LED0, LED1, LED2, LED3 (the `led1`–`led4` signals in the simulator window).
- Only **one** LED is on at a time.
- Each LED is on for **0.5 seconds**, then the next one turns on.
- After the last LED, the sequence starts over from the first.

The final effect should look like this:



A **reset button** is required, with the same behavior as in Task 2-1: pressed at any moment, the system returns to the initial state immediately.

Requirements:

- Use the **50 MHz** clock source as in Task 2-1.
- On the virtual board, map your reset signal to a **normal button** as in Task 2-1.

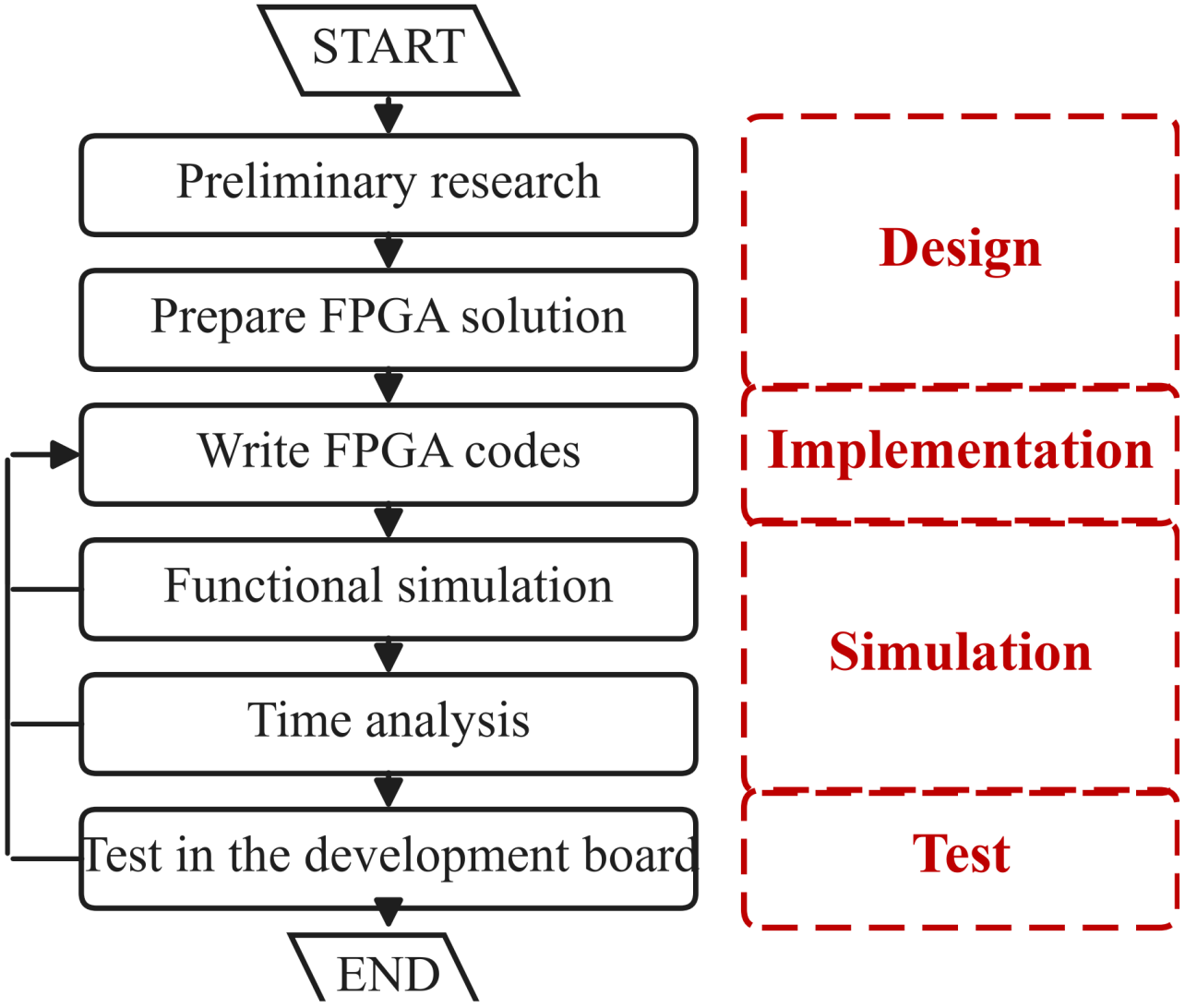
- You **may** use the shift operators `<<` / `>>` in your design; if you want to explore further, you **may** also compare their behavior with the arithmetic shifts `<<<` and `>>>`, but this is optional.

Verification: simulation **and** testing on the development board are both required (see Sections 3 and 4).

---

3. Required work — follow the development flow

Do every step below **for each task**, in order. The flow is the one from the lectures:



Whenever a verification step fails, **go back** — fix the code, or revisit the design — and come down the flow again. Skipping ahead with a known problem costs you more time than looping back.

Step 1: Preliminary research

Understand what you are asked to build before designing anything.

- Identify the experiment **objectives** and write down **every** functional requirement: the inputs, the outputs, and the required behavior for **every case**, including the initial state and a reset pressed at an arbitrary moment.
- Identify the **required devices** and analyze the **design constraints**, quantified where possible:
  - the **50 MHz** clock means a **20 ns** clock period — work out **how many clock cycles** make 0.5 s;

    - the electrical behavior of the buttons and LEDs (note: they are active-low on the development board);
    - how many pins and logic elements the FPGA chip provides — look it up in the **datasheet** and cite it.
- Do a short **literature review**: the FPGA datasheet, the tool documentation, and background reading on counters / shift registers. Record what you actually used — you will cite it in the report.

Output of this step: a requirement list in which **every item can be checked later** by a specific test.

Step 2: Prepare FPGA solution

Turn the requirements into a concrete, implementable design — on paper, before any code.

- Translate the requirements into a **design specification**: module name, port list (signal names, input/output, active level of each port), the internal variables (such as `cnt`) **with their bit widths justified**, and the **expected input/output waveforms** drawn before coding.
- Draw the **structure block diagram** with every signal labeled. Decide how many modules you need and why (a top-down design may still be a single module for these circuits — if so, justify it).
- Write down your **design decisions with reasons**, for example:
    - how do you measure 0.5 s with the 50 MHz clock, and why is your counter wide enough (and not wider than necessary)?
    - which reset style do you use — asynchronous or synchronous — and why?
    - for the running light: shift register, or counter plus decoding logic — and why?
    - which description style do you use (structural / dataflow / behavioral), and why is it appropriate here?
- Check **traceability**: every requirement from Step 1 must map to something in this design, and everything in the design must come from a requirement.

Step 3: Write FPGA codes

- Organize the project first: design codes in an `RTL/` folder, and testbench codes in a separate folder (e.g., `sim-wave/`).
- Write the Verilog following the coding rules in lectures: consistent naming, clean indentation, meaningful comments, synthesizable constructs only, no latches, no multiple drivers. Declare the bit width of every variable explicitly.
- **Verify the syntax** with iverilog: the code must compile cleanly.
- **Verify the implementation with yosys**: generate the RTL view and compare it against your block diagram — same ports, same logic, nothing extra; and read the resource numbers (wires, cells, memories) to see what was inferred. Warnings from iverilog or yosys usually mean real problems: fix them, do not ignore them.

Step 4: Functional simulation

Verify the **functions** of your design in simulation, before touching the board.

- Decide your **verification objectives first**: which cases must be checked for this design to be considered correct? Cover at least the initial state, the toggle moments, a reset pressed in the middle of a count and, for the running light, the wrap-around from the last LED back to the first.

- Write the testbench following the lecture: `reg` signals drive the inputs, `wire` signals observe the outputs, the **clock is generated** in the testbench, and the **reset stimulus is applied at deliberate times**. Keep the expected output of each step written next to its stimulus.
- The testbench must contain `$dumpfile` / `$dumpvars` so that a `.vcd` file is produced — GTKWave and the report figures both read it.
- Use GTKWave to **debug**: find where the actual behavior first deviates from your expectation. Watch the **internal variables** (such as `cnt`) as well, not only the ports.

Step 5: Time analysis

- Compare the simulated waveforms against the **expected waveforms you drew in Step 2**, moment by moment. They must be identical for every case in your verification plan; if not, locate the cause (design? code? testbench?) and loop back.
- Show evidence for the **exact timing**: the measured interval between two LED changes must equal the designed number of clock cycles. You may verify a full 0.5 s interval directly, or verify the logic with a shortened count and justify the full count by calculation — either way, the report must contain evidence for **both** the logic and the count.
- Give the expected and actual values of the **important internal variables** (such as `cnt` at the toggle moments and right after a reset), and explain any difference.
- State clearly what this analysis does **not** cover: the functional simulation is **zero-delay**. Real propagation delay, and whether the design still works under it, can only be confirmed on the board in Step 6.
- Produce the **waveform figures for your report** from the simulation data — **GTKWave screenshots are not acceptable in a report**. You may convert the VCD file to CSV with `vcd2csv.py` (get it here; it needs the `vcdvcd` Python package).

Step 6: Test in the development board

Required for **both** tasks.

- **Plan the pins**: map each module port to a board signal (`clk`, `B2`–`B5`, `led1`–`led5`, …); everything else stays unconnected. Remember Section 1: your reset goes to a **normal button**, not the `RESET` key. Write the mapping down — it goes into the report.
- **Verify the required behavior on the board, item by item**: the LED flashes (Task 2-1) or runs (Task 2-2); on and off alternate in the required order; pressing your reset button **at any moment** brings the system back to the initial state and restarts the timing. Record a **video of the board window** as your verification record; screenshots are optional supporting evidence.
- Note what the board can and cannot show: the board proves the **function**, while the **exact timing** is what you verified in simulation in Step 5.
- If the board behavior differs from simulation, debug and loop back to Step 3.

Know the limits of your tools

Each tool in this flow proves something different, and none proves everything: functional simulation is zero-delay and idealized; yosys performs *generic* synthesis, not a real FPGA implementation; the simulator models the real board but is not the real board — final confirmation always requires real hardware. In your report, show that you understand what each tool can and cannot guarantee.

---

4. Report requirements — one report per task

Write **one report for Task 2-1 and one for Task 2-2**. The report tells the story of your development flow; structure it as follows.

a) Introduction

- Background: what the circuit is (for 2-1: what a counter is and where it is used; for 2-2: what a running light is), and where such circuits appear in real systems.
- The purpose of this experiment.
- A brief overview of your design and your key results.
- What each group member contributed: use a list to show each member's contributions, with names and student IDs.

b) Requirements analysis

- The objectives and the **complete** functional requirements, including the initial state, the timing, and the reset behavior at an arbitrary moment.
- The design constraints, quantified: the clock period and the resulting **cycle count** for 0.5 s, the electrical behavior of the buttons/LEDs, the FPGA resources from the datasheet.
- The design specification you derived: ports with names, directions, and active levels; internal variables with bit widths; the expected waveforms.
- Make every requirement traceable — a reader should be able to follow each requirement to a spec item, a design element, and later a test case.

c) System design

- The structure block diagram, with every signal labeled.
- The expected input/output waveforms.
- Your design decisions and their justification (timing measurement and counter width, reset style, running-light architecture, description style).

d) Implementation

- Your final Verilog code, as readable text (monospace font), following the coding rules. **You cannot put all your code directly into this part.** Only show the idea of how you translated the design into code. **All code should be put on GitHub, with the link given in the report.**
- Evidence that the code passes the syntax check and yosys synthesis **without warnings**.
- The yosys RTL view, with your interpretation: does it match your block diagram? What do the `stat` numbers say? Any latches or unexpected logic?

e) Testing and results

- Your verification plan: which cases you test, and why those cases suffice.
- How your testbench works: the clock generation, the reset stimulus, the expected outputs, and how you compare expected vs actual. **You cannot put the testbench code directly into this part either; it goes on GitHub with the rest of the code.**
- The results: expected vs actual outputs for every case, **including the important internal variables** (such as `cnt`) at the key moments; explain any difference you observed.

- Waveform figures plotted from the simulation data (see Step 5), each numbered, captioned, and referenced in the text.
- The on-board verification: an itemized checklist of every case you tested on the development board. A **video recording** of the board window is **required** — upload it to a video website and give the link in the report; screenshots inserted in the report are optional supporting evidence.

f) Discussion *(optional but encouraged)*

- An honest discussion of the **limitations of the tools** you relied on (for example: zero-delay simulation vs real timing; what a shortened-count simulation does and does not prove).
- Bugs or problems you met: how you found them, what the waveform or RTL view showed, how you fixed them.
- Problems you could not solve in the given time, and your possible solutions.
- Interesting results or observations.

g) Conclusion

- Summarize the design and all verification results.
- Summarize the key discussion points, if any.

h) References

- Everything you cite, in **IEEE format** (IEEE Reference Guide) — the datasheet, tool documentation, and any tutorial you actually used.

Figure, table, and code standards

- Every figure and table is numbered, captioned, and referred to in the text.
- Waveforms are **plotted from data**. GTKWave screenshots are debugging aids and are not accepted as report figures.
- Board-verification records are the opposite: a **video link is required** (both tasks), and screenshots of the development-board window may be inserted as optional supporting evidence.
- Short code (a line or two) may appear as text, but not as screenshots. Longer code must be given as a GitHub link.

---

5. What to submit

The reports of **all tasks** must be combined into **one** PDF file. The report of **each** task must have **an individual cover page** with the task name, your names, and student IDs.

Plagiarism — of code or of report text — scores zero and is handled under the academic integrity rules.

If you use AI, you must comply with the **MUST AI Policy and Literacy** and the **Guidelines on Artificial Intelligence Application and Ethical Governance**, and state in your report which parts of the work were done with AI.